



University of Piraeus

School of Information and Communication Technologies

Department of Digital Systems

Level: Master's Programme of Studies

Wireless and Mobile Network Security

Case 3

Supervising Professors: Christos Xenakis and Georgios Efthymoglou

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

12/05/2026

Abstract

Compromise of the WEP security protocol, highlighting its inherent vulnerabilities, particularly in the management of the Initialization Vector (IV). Through a controlled experimental environment in Kali Linux, a virtual access point was created and a client device was utilized to generate network traffic. Subsequently, the aircrack-ng tool was employed utilizing the PTW analysis method, achieving the complete recovery of the cryptographic key in minimal time.

Contents

Introduction.....	1
Practical part of the assignment.....	2
Bibliography.....	10

Introduction

In this assignment, the process of compromising the WEP (Wired Equivalent Privacy) security protocol is studied and analysed in practice. WEP was introduced in 1997 to provide wireless networks with a level of confidentiality equivalent to that of wired networks. It relies on the RC4 symmetric encryption algorithm for data confidentiality and on the CRC-32 algorithm for integrity checking (ICV). Despite its widespread use in the past, WEP proved extremely vulnerable due to serious design weaknesses. The protocol's fundamental weakness lies in the management of the Initialization Vector (IV). The IV is a small 24-bit auxiliary number added to the network's fixed key to make the encryption of each packet unique. The problem is that, on the one hand, the IV is transmitted openly in plaintext together with the packet and, on the other hand, due to its small size, the available numbers are exhausted very quickly on an active network. This forces the router to reuse the same IVs. When an IV is reused, the encryption becomes predictable, allowing an attacker who captures the traffic to extract the original Wi-Fi password. In addition, the RC4 algorithm exhibits weaknesses that allow the creation of 'weak keys'. Exploiting these gaps, an attacker can collect a sufficient number of encrypted packets and recover the network's original key in minimal time. The purpose of this exercise is to build an experimental WEP environment and to practically perform attacks to recover the key, highlighting the above vulnerabilities.

For personal-data protection purposes and following security best practices when publishing this report, the network's sensitive identifying details have been modified. Parts of the physical addresses (MAC Addresses) and the internal IP addresses of the devices used in the exercise have been intentionally obscured in the screenshots.

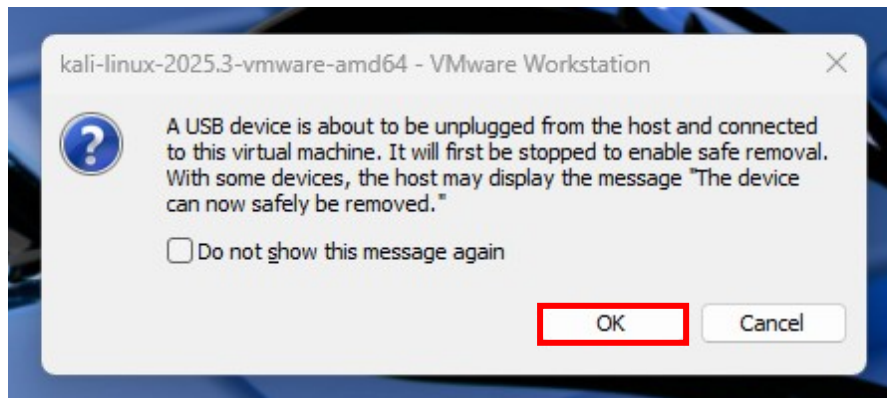
Practical part of the assignment



Regarding the first step of connecting the equipment to VMware, the process begins by plugging the TP-LINK TL-WN722N v4 (150 Mbps) wireless USB network adapter with a detachable antenna into a free USB port of the laptop. Then, through VMware's main menu (at the top of the screen), we navigate to 'VM > Removable Devices' and locate this device in the list, which is usually shown by the manufacturer's name. Finally, by selecting the 'Connect (Disconnect from host)' command, the card is disconnected from the host operating system (Windows) and assigned exclusively for use within the Kali Linux environment, becoming the main tool for the successful completion of the assignment.

When the device is connected, the VMware environment displays a standard informational message confirming that the TP-Link network card is released from the host machine (Windows) and attached exclusively to the Kali Linux virtual machine.

In
to
the



order
verify

successful recognition of the wireless interface by the operating system, a terminal window is then launched within Kali Linux and the `iwconfig` command is executed. As shown by the command's results, the Kali Linux operating system successfully recognised the external network adapter, assigning it the interface name `wlan0`. At the same time, it was found that the card's default operating Mode was 'Managed'. In order to switch to Monitor Mode without errors, it was deemed necessary to terminate any system processes and network services (such as NetworkManager) that might cause interference or attempt to arbitrarily reset the card to its initial state. For this purpose, the command `sudo airmon-ng check kill` was executed, thereby ensuring the smooth preparation of the wireless interface for the next stages of the analysis.

```
(kali㉿kali)-[~]
└─$ sudo airmon-ng check kill
[sudo] password for kali:
Killing these processes:

    PID Name
    2347 wpa_supplicant

(kali㉿kali)-[~]
```

During the execution of the previous command, the system successfully detected and terminated the `wpa_supplicant` process, which is responsible for managing standard wireless connections in the background. Having secured the appropriate environment and eliminated potential interference, the `wlan0` network card was switched to Monitor Mode via the command `sudo airmon-ng start wlan0`. Finally, to definitively confirm the successful change of the interface's operating mode, the `iwconfig` command was executed again.

Switching the TP-Link wireless card to Monitor Mode is a fundamental step for carrying out the attack. Unlike the default mode (Managed Mode), where the card processes only the packets destined for itself, Monitor Mode allows the capture of all wireless traffic (raw frames) on the broadcast channel, making it possible to intercept the encrypted WEP packets.

Since modern home routers no longer support the obsolete WEP protocol for security reasons, it was necessary to create a virtual access point within the Kali Linux environment. This virtual network will act as the target of the attack, providing the experimental environment needed to collect the IVs and subsequently recover the cryptographic key.

The choice of the `airbase-ng` tool to create the 'JK_WEP' virtual network highlighted a critical technical parameter regarding the format of the cryptographic key. Unlike commercial routers that allow entering passwords in ASCII format (e.g. 5 characters), `airbase-ng` requires the key to be defined strictly in hexadecimal format via the `'-w'` parameter. This requirement stems from the architecture of 64-bit WEP. The total key is derived from the sum of a fixed 40-bit key and a 24-bit IV. Since each hexadecimal digit corresponds to 4 bits, the 40-bit key must consist of exactly 10 hexadecimal digits (0-9, A-F). The initial use of a 5-digit ASCII password led to a key-length error ('Invalid WEP key length'), making it necessary to correct the command.

The virtual network was successfully started with the following command, which remained active in a separate terminal throughout the attack: `sudo airbase-ng -e "JK_WEP" -c 6 -W 1 -w 1122334455 wlan0`. Upon executing the command, the

virtual interface `at0` was created and the Access Point began broadcasting with the MAC address of the TP-Link card as its BSSID.

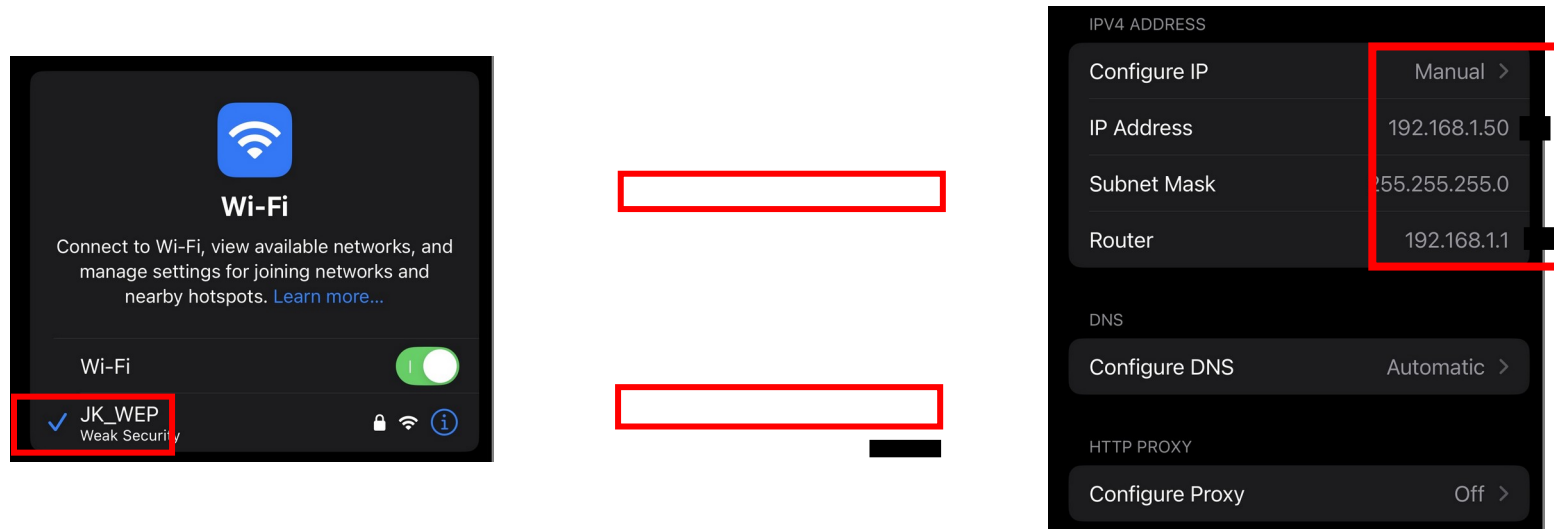
```
(kali㉿kali)-[~]
└─$ sudo airbase-ng -e "JK_WEP" -c 6 -W 1 -w 1122334455 wlan0
[sudo] password for kali:
16:31:20 Created tap interface at0
16:31:20 Trying to set MTU on at0 to 1500
16:31:20 Access Point with BSSID 3C:52:A1: started.
16:32:09 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:32:09 SKA from 90:EC:EA:
16:32:09 Client 90:EC:EA: associated (WEP) to ESSID: "JK_WEP"
16:32:23 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:32:23 SKA from 90:EC:EA:
16:32:23 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:32:35 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:32:35 SKA from 90:EC:EA:
16:32:35 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:32:35 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:33:08 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:08 SKA from 90:EC:EA:
16:33:08 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 SKA from 90:EC:EA:
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:11 SKA from 90:EC:EA:
16:33:11 Client 90:EC:EA: associated (WEP) to ESSID: "JK_WEP"
16:33:16 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:16 SKA from 90:EC:EA:
16:33:16 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:33:48 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:48 SKA from 90:EC:EA:
16:33:48 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:48 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:48 SKA from 90:EC:EA:
16:33:48 Client 90:EC:EA: reassociated (WEP) to ESSID: "JK_WEP"
16:33:59 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
16:33:59 SKA from 90:EC:EA:
16:33:59 Client 90:EC:EA: associated (WEP) to ESSID: "JK_WEP"
16:34:04 Broken SKA: 90:EC:EA: (expected: 178, got 165 bytes)
```

In order to simulate a user's activity on the network and to start generating traffic for packet collection, an iPhone terminal device was used as the 'target' (client). During the process of connecting to the 'JK_WEP' virtual network, significant limitations arose due to the modern security safeguards of the operating systems:

- Protocol authentication (Broken SKA): Initially, an inability to complete the connection was observed, with the 'Broken SKA' error. This indicates the failure of the Shared Key Authentication method, which requires the exchange of an encrypted challenge. The problem was solved by switching to the Open System Authentication method, which allowed the device to associate successfully, despite the operating system's 'Weak Security' warning.

- Manual network configuration (Static IP): Since the virtual access point (airbase-ng) did not provide a DHCP service for automatic address assignment, the device was unable to complete the connection. To resolve the issue, a static IP address was manually assigned in the client's Wi-Fi settings (IP: 192.168.1.5XX, Subnet Mask: 255.255.255.0, Gateway: 192.168.1.XXX).

After the successful connection, repeated attempts were made to access websites through a browser. Although the network did not provide internet access, this action forced the device to continuously transmit encrypted data packets over the air. These packets constituted the raw material for collecting the IVs required for the final analysis.



With the terminal device successfully associated with the virtual network, the process of targeted capture of the wireless traffic on channel 6 began. For this purpose, the airodump-ng tool was used, configured to save the collected packets in .cap files for later analysis, using the command `sudo airodump-ng --bssid 3C:52:A1:XX:XX:XX -c 6 -w wep_capture wlan0`.

The primary goal was the rapid collection of a sufficient number of IVs. Initially, the classic ARP Replay Attack technique was attempted via the aireplay-ng tool. However, during the experimental procedure it was found that injecting packets at an extremely high frequency overloaded the wireless network card. This resulted in instability of the interface and repeated disconnection of the client, making data collection ineffective.

In order to ensure the stability of the connection and the uninterrupted flow of data, an alternative, controlled method of traffic generation was adopted, leveraging the virtual interface at0 created by the airbase-ng tool. As a first step, it was necessary to bring the interface up and assign it the router's IP address with the command `sudo ifconfig at0 192.168.1.XXX up`. Then, a controlled transmission of ICMP packets was carried out towards the static IP of the target device, via the command `sudo ping -i 0.1 192.168.1.5XX`. Using the `-i 0.1` parameter, exactly 10 packets per second were generated. This approach allowed the steady and linear collection of thousands of IVs without crashing the wireless interface or disconnecting the client, providing the material needed for the final phase of the analysis.

The final phase of the attack focused on analysing the collected data using the aircrack-ng tool. The program was configured to analyse the entire set of recorded files (`wep_capture*.cap`) in combination, in order to maximise the volume of available data, with the command `sudo aircrack-ng wep_capture*.cap`. Although the algorithm's statistical analysis started automatically once 5,000 IVs were reached, the randomness of the generated packets did not immediately yield the necessary mathematical collisions. Therefore, capturing had to continue until approximately 15,000 IVs were collected. As shown in the screenshot below, with this volume of data the PTW (Pyshkin, Tews, Weinmann) attack managed to successfully correlate the mathematical repetitions of the IVs and recover the 64-bit WEP key 11:22:33:44:55 with 100% success.

It is noteworthy that the use of the PTW (Pyshkin, Tews, Weinmann) cryptanalysis method by the aircrack-ng tool dramatically reduced the amount of data required. Unlike older attacks (e.g. Fluhrer, Mantin, Shamir) that required hundreds of thousands of packets, the PTW method recovered the key with only ~15,000 IVs, exploiting the statistical weaknesses of the RC4 algorithm more efficiently.

Conclusion

The practical exercise demonstrated that the WEP protocol is completely insecure for modern environments. The ability to recover the key in less than 10 minutes, even using a virtual environment, highlights the critical importance of using modern standards such

as WPA3. The difficulties encountered with the iPhone terminal devices also underline the built-in security safeguards of modern operating systems against obsolete protocols.

Bibliography

- Recommended settings for Wi-Fi routers and access points - Apple Support. (2026, March 26). Retrieved from <https://support.apple.com/en-us/102766>
- airbase-ng [Aircrack-ng]. (2018, March 11). Retrieved from <https://www.aircrack-ng.org/doku.php?id=airbase-ng>
- Fluhrer, S., Mantin, I., & Shamir, A. (2001). Weaknesses in the key scheduling algorithm of RC4. In *Selected Areas in Cryptography: 8th Annual International Workshop, SAC 2001 Toronto, Ontario, Canada, August 16-17, 2001. Revised Papers* (pp. 1-24). Berlin, Heidelberg: Springer. Retrieved from https://www.cs.cornell.edu/people/egs/615/rc4_ksaproc.pdf
- Tews, E., Weinmann, R. P., & Pyshkin, A. (2007). Breaking 104 bit WEP in less than 60 seconds. In *Information Security Applications: 8th International Workshop, WISA 2007, Jeju Island, Korea, August 27-29, 2007, Revised Selected Papers* (pp. 188-202). Berlin, Heidelberg: Springer. Retrieved from <https://eprint.iacr.org/2007/120.pdf>
- Bedi, C., & Vaja, A. (2014). *Kali Linux wireless penetration testing beginner's guide*. Birmingham, UK: Packt Publishing Ltd. Retrieved from https://api.pageplace.de/preview/DT0400.9781783280421_A24763783/preview-9781783280421_A24763783.pdf